Capacity and training duration for a small causal GPT

DASE7506 - MP1 Small Language Model Challenge | Experiment report | 29 September 2026

1. Objective and main finding

This study tests whether the supplied small GPT benefits from additional optimization and increased hidden width under a fixed lightweight evaluation protocol. The selected predictor increases width from 128 to 192 and trains from random initialization for 3,600 updates. A residual-dropout implementation is investigated but rejected on validation evidence; dropout is zero in the submitted model. No new architecture family is claimed.

Full-test BPB	CPU scoring ratio	Peak evaluation RAM
1.722827	1.229 x baseline	2.117 GiB

The original baseline scores 2.101265 test BPB; the final predictor scores 1.722827, a reduction of 0.378438 BPB (18.01%). This total combines training-duration and width changes. At the same 29,491,200 training targets, increasing width lowers mean validation BPB from 1.793761 to 1.701554 across seeds 17 and 42, a 5.14% reduction. The two paired improvements are 0.092101 and 0.092312 BPB.

Protocol and data boundary

The unchanged course scorer implements protocol 7506-mp1-wt2-v2 on supplied WikiText-2 text and a fixed train-fitted BPE tokenizer with 2,048 entries. Each independent causal window contains at most 256 scored targets; the final short window is included. Every target after the first token of a split is scored once. BPB equals summed negative natural-log probability divided by ln(2) and the entire split UTF-8 byte count. Validation contains 376,599 targets and 1,148,007 bytes; test contains 428,405 targets and 1,292,013 bytes.

Parameter updates use only training text. Validation selects configurations and the final seed. The supplied loader reads all three splits, but the training code consumes training tokens for gradients and validation tokens for checks; it does not use test targets for updates or selection. Final test evaluation follows the recorded freeze. There are no external weights, text, retrieval assets, cached answers, evaluation network calls or persistent cross-window state.

Evidence scope

All numerical results are taken from supplied run/score files and independently checked when packaging. Raw logs, per-window losses, hashes and exact commands are included. Student ID3036707707 are not inferred; the course submission form must identify the author. Hardware: Apple M5 Max, 18 CPU cores, 36 GB RAM, macOS 26.4. Recorded training/evaluation uses Python 3.12.10,
PyTorch 2.7.1, CPU FP32 and four threads.

2. Model and training method

Baseline and selected capacity change

The baseline has four pre-normalized causal Transformer blocks, learned token/position embeddings, GELU feed-forward layers of width 4d, LayerNorm and tied input/output embedding weights. The selected model preserves these components, depth, vocabulary and context while changing hidden width d from 128 to 192. Four attention heads are retained, so head dimension changes from 32 to 48; feed-forward hidden width changes from 512 to 768. Parameter count rises from 1,088,256 to 2,222,208 (2.042x). The width experiment therefore tests a coupled capacity increase, not isolated head size or isolated MLP width.

Residual dropout as a rejected mechanism

The student implementation inserts nn.Dropout after the attention output projection and after the MLP output, before each residual addition. At training probability p=0.1, values are randomly zeroed with PyTorch inverted-dropout scaling; evaluation disables dropout. With p=0, the forward computation matches the original baseline. This mechanism was proposed to improve generalization, but the observed validation curves did not show a deterioration that established overfitting, and dropout worsened results at both tested training budgets. The selected checkpoint uses p=0.

Optimization recipe

Setting	Value
Initialization	Random normal weights, standard deviation 0.02; no parent checkpoint
Optimizer	AdamW; peak learning rate 0.001; weight decay 0.1; PyTorch defaults otherwise
Batch and target count	32 sequences x 256 targets = 8,192 targets/update
Schedule	100-update warmup multiplied by cosine decay with 0.1 floor
Stability / precision	Gradient norm clipped to 1.0; FP32 training and scoring
Budgets	1,200 updates: 9,830,400 targets; 3,600: 29,491,200 targets
Selection	Validation only; final checkpoints, not unsaved intermediate minima

The trainer samples random contiguous training segments. Its learning rate at zero-based step t is 0.001 x min(1,(t+1)/100) x [0.1 + 0.45 x (1 + cos(pi x t/T))], where T is total updates. Increasing T changes both exposure and the learning-rate trajectory. Consequently, the short-versus-long comparison measures a longer training recipe, not extra updates under an otherwise identical stepwise schedule.

The selected run uses seed 42, width 192, 3,600 updates and dropout 0. Seeds 17 and 42 are paired across widths. The model was selected using its final validation BPB of 1.697728808551342. All runs start from scratch; cumulative search cost counts every run, including controls and unsuccessful dropout trials.

3. Recorded experiments and validation curves

Run role	Seed	Width	p	Steps	Val. BPB	Train s
Original baseline	17	128	0	1,200	2.071087	173.95
Implementation control	17	128	0.0	1,200	2.071087	176.16
Dropout trial	17	128	0.1	1,200	2.104548	192.34
Long control	17	128	0.0	3,600	1.797481	524.92
Long dropout trial	17	128	0.1	3,600	1.839950	570.66
Second-seed control	42	128	0.0	3,600	1.790041	521.61
Wide model	17	192	0.0	3,600	1.705380	732.84
Selected model	42	192	0.0	3,600	1.697729	712.78
Installation/speed check	17	128	0	100	3.035893	14.57

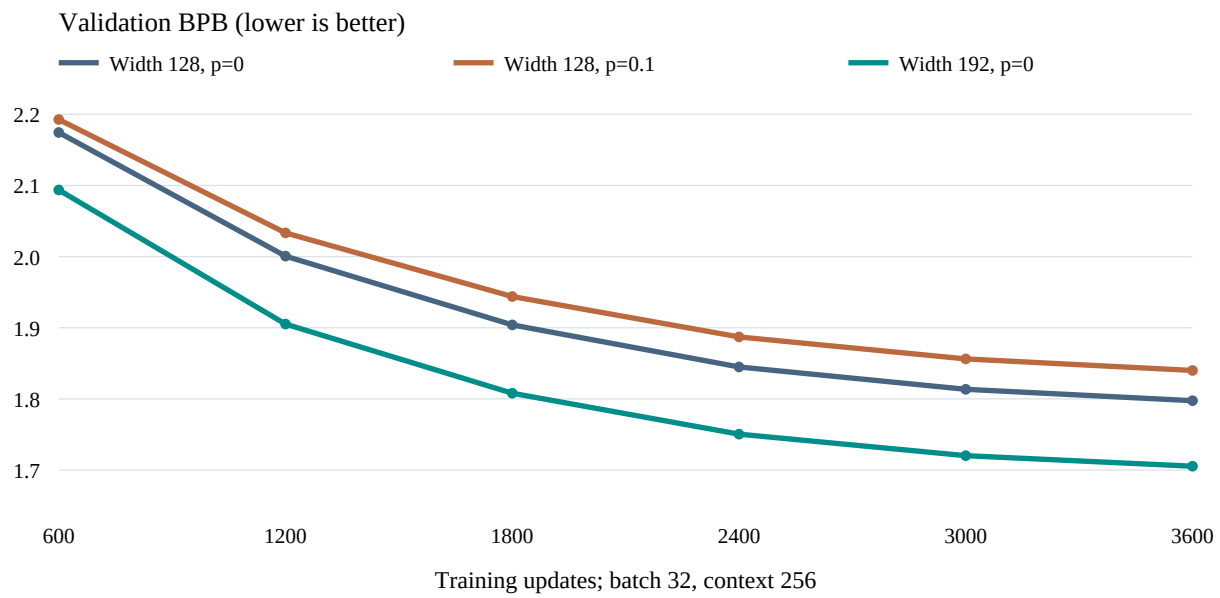

Figure 1. Validation learning curves for seed 17 under the 3,600-update schedule. Each plotted point comes from the saved validation_history. All three curves decrease at the recorded checkpoints; this does not rule out behavior between checkpoints or under other training budgets. The final checkpoint is retained by the unmodified trainer.

The 100-step run is a pipeline/speed check, not a competitive baseline. The implementation control exactly reproduces the baseline validation BPB at the reported precision. All table entries are validation results; test scores are reported separately.

4. Controlled comparisons and critical analysis

Capacity ablation at equal processed targets

Seed	Width 128 BPB	Width 192 BPB	Reduction
17	1.797481	1.705380	0.092101
42	1.790041	1.697729	0.092312
Mean	1.793761	1.701554	0.092207 (5.14%)

Each paired comparison processes exactly 29,491,200 training targets with identical depth, heads, optimizer recipe, batch size, training duration and dropout setting. Reverting width to 128 ablates the selected capacity increase and also supplies the required equal-target control. Two seeds give consistent direction and similar effect size, but are too few to establish a broad statistical significance claim.

A plausible interpretation is that the wider representations and larger feed-forward layers better fit training-derived predictive structure. This is an interpretation, not a directly identified causal explanation: head dimension, parameter count and compute change together, and no equal-FLOP or equal-wall-time control was run. The work establishes an empirical quality/cost trade-off within this model family.

Training duration and dropout

For width 128, seed 17 and p=0, extending the recipe from 1,200 to 3,600 updates lowers validation BPB from 2.071087 to 1.797481 (0.273606, 13.21%). The increase in training targets and the changed cosine schedule are inseparable here. At 1,200 updates, p=0.1 worsens BPB by 0.033461; at 3,600 it worsens BPB by 0.042469. The tested regularization strength is not supported for the selected budgets. The experiment does not exclude weaker dropout or other regularizers.

Cost of the selected mechanism

Mean training time across the paired seeds increases from 523.26 s to 722.81 s (38.14%) when widening the model. Parameters increase by 104.20%, while mean validation BPB falls by 5.14%. Equal token exposure should not be confused with equal computational effort. The selected seed-42 run alone takes 712.78 s of training; its selection from two seeds is disclosed rather than presented as a pre-specified seed.

Search accounting

The nine recorded completed runs process 177,766,400 targets. Summed training time is 3619.84 s (60.33 min). Summed full training-process time is 3844.06 s (64.07 min), including preparation and validation. The four original standalone score calls contribute 17.35 scoring seconds, excluding process overhead. Packaging reproduction is recorded separately. Installation, human work, electricity and any deleted/interrupted runs are not quantified; additional runs, if any, require disclosure.

5. Frozen full-test result and resource verification

Metric	Original baseline	Selected predictor
Full-test BPB	2.1012651958438786	1.7228267184138215
Scoring seconds	4.144501	5.094313
CPU scoring ratio	1.000	1.229174 (limit: 5)
Final predictor peak RSS	-	2.116852 GiB (limit: 4)
Checkpoint bytes	-	8,906,741 (8.494130 MiB)
Conservative inference assets	-	8.620992 MiB (limit: 64)

Both score calls use the complete test split, CPU FP32, four threads and the unchanged official evaluator. The scoring timer excludes installation, model loading and tokenization. Peak RAM is whole-process maximum resident set size from macOS /usr/bin/time -l, reported in bytes; it includes loading and data preparation. Zero GPU-memory fields in JSON are not used as RAM evidence. Original timings are single observations, not confidence intervals or guaranteed times on other machines.

The conservative final-asset inventory includes the checkpoint, model and scorer source, fixed tokenizer, configuration and requirements. There is no additional retrieval or memory asset. The separately supplied baseline checkpoint is a comparator only. Benchmark text is evaluation input, not learned inference assets. Both model and data checksums are retained.

Freeze and independent directory reproduction

The freeze record is dated 29 September 2026 at 14:10:30 UTC (22:10:30 UTC+8). The documented workflow selects the final predictor on validation and performs test scoring afterwards. Hashes verify the identity of the supplied final checkpoint, implementation, evaluator and tokenizer. Hashes alone cannot prove the full history of unseen development actions.

Packaging checks run from the independent deliverable code directory, using the existing exact-version course Python environment. Both full-test BPBs reproduce exactly: baseline 2.1012651958438786, final 1.7228267184138215. The five supplied contract tests pass. This is same-machine packaged-code reproduction, not a clean dependency installation or a cross-platform verification.

The successful packaging resource check records final scoring 5.534851 s, a 1.272 ratio to its paired baseline, and peak child-process RSS 2.304 GiB using Python resource.getrusage. Initial /usr/bin/time -l wrappers could not query kern.clockrate in the tool sandbox although scoring completed; these attempts and the successful checks are retained separately. They do not replace the original measurements.

Full-test losses contain 1,674 independent windows. Re-summing supplied per-window negative log likelihoods reconstructs the stated BPB within floating-point summation tolerance. This verifies score-file consistency, not the absence of every possible methodological error.

6. Reproduction, limitations and attribution

Reproduce without retraining

Extract mp1-code and mp1-checkpoints next to each other. In mp1-code, create a Python 3.12 virtual environment, install requirements.txt and run scripts/verify_release.py. Then run the unchanged evaluator using the final checkpoint:

```
python evaluate.py --checkpoint ../mp1-checkpoints/checkpoint.pt
--device cpu --precision fp32 --threads 4 --split test
--output reproduced/final-test.json
```

Join the three displayed lines into one command. README.md gives complete copyable installation, training, scoring and memory-measurement commands. evidence/training_commands.json specifies every recorded run. No retraining, optimizer state or network access is required for scoring after obtaining dependencies and the two archives.

Limitations and justified claims

This is a small controlled configuration study, not a novel architecture. The capacity comparison spans two seeds, one dataset/tokenizer and one paired training budget. It does not identify the best possible model, establish a leaderboard rank or guarantee a grade. The submitted model combines longer training and width scaling; test improvements cannot isolate either factor. Model selection among configurations and seeds may favor validation, and only one frozen predictor is ultimately submitted. Resource results are machine-dependent and use the course limits as stated; the supplied README leaves detailed timing repetition rules unspecified.

The evidence supports rejecting p=0.1 at the tested budgets and adopting width 192 for its measured validation gain at a modest observed scoring-time increase. It does not show that all regularization is harmful, that width alone explains the total test gain, or that larger models would remain beneficial within the resource budget.

AI assistance and reused implementation

OpenAI Codex provided assistance with assignment interpretation, residual-dropout code, command generation and， result interpretation. The student executed the recorded training workflow; Codex ran later packaging checks. The GPT,training recipe, dataset, tokenizer, evaluator and contract tests are reused from the DASE7506 starter; the source attribution is preserved in the repository.